

KoNLPy 형태소 분석기의 불규칙 용언 처리 성능 분석
- komoran과 okt 클래스 비교를 중심으로

과목명: 논리적 사고와 글쓰기

담당 교수: 이미나 교수님

제출일: 2024년 6월 16일

제출자: 홍익대학교 컴퓨터공학과

C411161 정지윤

<목 차>

1. 서론
2. 텍스트 전처리 개념
 - 2.1 텍스트 전처리의 의미
 - 2.2 텍스트 전처리의 중요성
3. KoNLPy의 komoran과 okt클래스
 - 3.1 KoNLPy 라이브러리
 - 3.2 komoran 클래스
 - 3.3 okt 클래스
4. komoran과 okt클래스를 이용한 불규칙 용언의 형태소 분석 연구
 - 4.1 연구 설계 및 절차
 - 4.2 분석 결과 정리
5. 분석 결과
 - 5.1 komoran 클래스 분석 결과
 - 5.2 okt 클래스 분석 결과
 - 5.3 komoran과 okt 클래스의 분석 결과 비교
6. 결론

1. 서론

“우리는 이미 일상생활에서 챗봇, 인공지능 스피커, 검색엔진 등과 같은 다양한 자연어 처리 기술을 경험하고 있다.”¹⁾ 이러한 자연어 처리 기술들은 컴퓨터가 복잡한 자연어를 효과적으로 이해하고 처리할 수 있도록, 텍스트 전처리 과정을 거친다. “토큰화는 텍스트 전처리의 가장 기초적인 작업으로, 텍스트를 토큰이라는 단위로 나누는 작업이다.”²⁾ 한국어 텍스트의 토큰화는 정확도를 높이기 위해 주로 형태소 단위로 수행된다. 형태소 분석 결과는 자연어 처리 기술의 성능에 영향을 주기 때문에 정확하게 수행할 필요가 있다.

“현재, Konlpy 라이브러리에서 제공하고 있는 한국어 형태소 분석기는 총 5개로, Komoran, Okt, Hannanum, Kkma, Mecab이 있다.”³⁾ 각 형태소 분석기는 성능과 결과가 다르기 때문에, 프로그램의 개발자는 용도에 맞는 적절한 형태소 분석기를 선택해서 사용해야 한다.⁴⁾

그러나, 교착어인 한국어에는 형태소 분석을 어렵게 만드는 요소들이 있다. 그 중 하나가 불규칙 용언이다. 따라서 각 형태소 분석기가 용언의 활용형을 정확하게 분석하는지 연구할 필요가 있다. 이에 필자는 본 연구를 통해 konlpy의 komoran과 okt 클래스를 비교·분석하여 불규칙 용언의 형태소 분석에 가장 적합한 클래스를 도출하는 것을 목적으로 한다.

기존의 konlpy 연구들은 불규칙 용언의 형태소 분석에 대한 논의가 미흡하다. 또한, 대부분의 기존연구는 형태소 분석기의 처리속도나 정확도에 초점을 맞추었다. 이에 필자는 메모리 사용의 효율성 측면에서 komoran과 okt 클래스를 비교함으로써 본 연구를 차별화하고자 한다.

2. 텍스트 전처리의 개념

2.1 텍스트 전처리의 의미

“자연어 처리(Natural Language Processing)는 컴퓨터가 인간의 언어인 자연어를 이해하고 처리하는 인공지능의 분야이다.”⁵⁾ “우리는 이미 일상생활에서 챗봇, 인공지능 스피커, 검색엔진 등과 같은 다양한 자연어 처리 기술을 경험하고 있다.”⁶⁾ 이러한 자연어 처리 기술은 컴퓨터에서 자연어를 효과적으로 처리할 수 있도록 반드시 텍스트 전처리 과정을 거친다. 컴퓨터는 구조가 복잡한 자연어를 직접적으로 처리하기는 어렵기 때문이다. 텍스트 전처리 과정을 통해 자연어는 컴퓨터가 처리하기 쉬운 형태로 변환된다.

“텍스트 전처리(Text preprocessing)는 자연어 처리의 초기 단계로, 입력된 텍스트를 수치 형태로 변환하여 컴퓨터가 처리할 수 있는 형태로 만드는 과정이다.”⁷⁾ 이는 토큰화, 정제 및 추출, 인코딩 등의 순서로 수행된다.

1) 김철희 외, 「한국어 교육을 위한 형태소 분석기의 조건과 향후 과제」, 한국국어교육학회, 2021, p.2

2) *Ibid.*, p.4.

3) 최아리, 「자연어처리 기술을 활용한 국내 형사 판결문 검색 모델 비교 연구」, 한림대학교 대학원, 2022, p.57

4) *Ibid.*, p.58

5) 한국정보통신기술협회, 「지식백과 자연어 처리」, 2024년 6월 8일 검색

<https://terms.naver.com/entry.naver?docId=5917571&cid=66682&categoryId=66682>

6) 김철희, *loc. cit.*

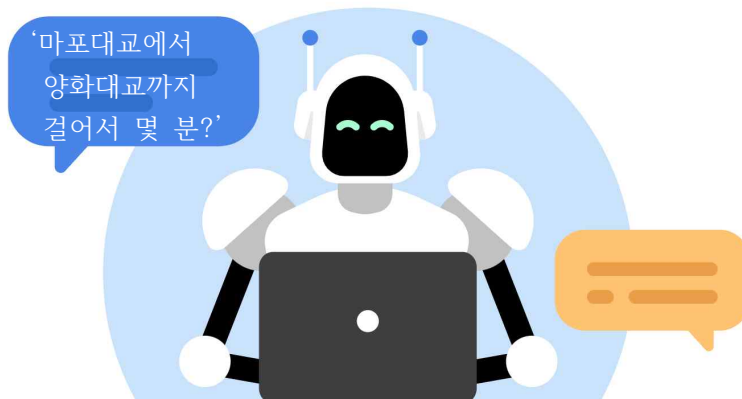
7) 황현선 「한국어 자연어처리를 위한 딥러닝 모델의 속도 향상」, 강원대학교 대학원, 2022, p.15

2.2 텍스트 전처리의 중요성

“토큰화는 텍스트 전처리의 가장 기초적인 작업으로, 텍스트를 토큰이라는 단위로 나누는 작업이다.”⁸⁾ 토큰은 문장, 단어, 형태소 등이 될 수 있다. 한국어 텍스트의 토큰화는 정확도를 높이기 위해 주로 형태소 단위로 수행된다.⁹⁾ “형태소는 국어에서 의미를 가지는 최소 단위로, 형태소 분석은 문장을 형태소 단위로 나누는 과정이다.”¹⁰⁾ 이때, 형태소 분석 과정을 수행하는 도구를 ‘형태소 분석기’라고 한다.

부정확한 형태소 분석은 후속단계인 정제 및 추출, 인코딩 단계에 직접적으로 영향을 주어 여러 문제를 유발한다. 자연어 처리 기술 중 하나인 챗봇에서 텍스트 전처리의 중요성을 보여주는 예시를 보자. 교통정보를 제공하는 챗봇의 검색창에 ‘마포대교에서 양화대교까지 걸어서 몇 분’을 입력하는 경우이다. 텍스트 전처리 이전의 컴퓨터는 ‘마포대교에서 양화대교까지 걸어서 몇 분’이라는 자연어로 이루어진 검색어를 직접적으로 이해하지 않는다. 검색어는 토큰화, 정제와 추출, 인코딩이라는 일련의 텍스트 전처리 과정을 거쳐 컴퓨터가 이해할 수 있는 형태로 변환된다. 이후 챗봇이 일련의 작업들을 수행해서 사용자에게 적절한 정보를 제공한다.

‘ㄷ’ 불규칙 용언인 ‘걸어서’의 올바른 형태소 분석은 [‘걸’, ‘어’, ‘서’]이다. [‘걸’, ‘어’, ‘서’]로 부정확하게 형태소 분석되는 경우 생기는 문제들이 있다. 첫째, 데이터의 의미가 왜곡될 수 있다. ‘걸어서’가 ‘옷장에 옷을 걸다’의 의미로 왜곡되는 경우 사용자가 관련성이 낮은 정보를 얻을 수 있다. 둘째, 메모리 소모량과 처리 시간 측면에서 비효율적이다. 예를 들어, 부정확하게 분석된 형태소가 많은 경우, 정제 및 추출과정에서 이를 수정하기 위한 추가적인 연산들이 누적되면서 처리시간이 과도하게 증가할 수 있다. 이는 프로그램의 전반적인 성능저하를 유발한다.



<챗봇>

8) 김철희, *loc. cit*

9) OpenAI(2022). ChatGPT(November 30 version) [Large language model], <https://openai.com/index/chatgpt>

10) 구혜승, 「한국어 에세이 자연어 처리를 위한 한국어 형태소 분석기 개선 방안」, 한국교원대학교 대학원, 2024. p.13

3. KoNLPy의 komoran과 okt 클래스

3.1 KoNLPy 라이브러리

“Python konlpy 라이브러리는 최근 자연어 처리 기술에서 많이 사용되는 오픈소스 중 하나로, C나 Java 등 다양한 언어로 개발된 한국어 형태소 분석기를 python에서 제공한다.”¹¹⁾ “형태소 분석기는 대체로 사람이 형태소 분석을 해놓은 텍스트 데이터를 학습하여 만들어진 다.”¹²⁾ 현재 konlpy에서 사용가능한 한국어 형태소 분석기에는 Komoran, Okt, Hannanum, Kkma, Mecab이 있다. Konlpy 형태소 분석기는 일반적으로 교착어인 한국어의 패턴을 잘 반영하여 형태소 분석을 수행한다. “그러나 각 형태소 분석기마다 분석 성능에 차이가 있고, 분석의 기준도 조금씩 다르다.”¹³⁾ 본 연구에서는 konlpy를 ‘from konlpy.tag import Komoran, Okt’의 형식으로 가져와서 사용한다.

3.2 komoran 클래스

komoran은 현재 오픈소스로 공개되어있는 한국어 형태소 분석기이다. “이 형태소 분석기는 한국어 텍스트를 형태소 단위로 분석하여 단어의 품사를 태깅하고, 구문 분석 및 의미 분석에 활용된다.”¹⁴⁾ komoran은 java로 구현되었으며, 정확성이 높고 속도가 빠르다. komoran은 통계적 분석, 정보 검색 등 다양한 자연어 처리 작업에 사용된다. komoran은 한국어 텍스트를 형태소 단위로 분석하고 품사를 태깅하는 pos(), morphs(), nouns() 메소드를 제공한다. 본 연구에서는 (형태소, 품사)의 튜플을 요소로 가지는 리스트를 반환하는 pos() 메소드를 사용한다.¹⁵⁾

3.3 okt 클래스

okt는 트위터에서 개발한 한국어 형태소 분석기로, 주로 소규모 데이터나 소셜 미디어 데이터 분석에 적합하다. “사용자가 원하는 단어를 추가할 수 있는 사용자 정의 사전 기능을 제공한다.”¹⁶⁾ 이를 통해 분석 결과의 정확도를 높일 수 있고, 간편하게 사용가능하다는 장점이 있다. komoran과 마찬가지로 pos(), morphs(), nouns(), 메소드를 제공한다.

11) 최아리, *op. cit.*, p.57

12) 서울대학교 AI연구원, 「지식백과 형태소 분석기」, 2024년 6월 10일 검색,
<https://terms.naver.com/entry.naver?docId=6653681&cid=69974&categoryId=69974>

13) *Ibid.*

14) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

15) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

16) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

4. 불규칙 용언의 형태소 분석 비교 연구

4.1 연구 설계 및 절차

본 연구는 실험 연구로 진행한다. 연구대상은 용언의 불규칙 활용형에서 어간이 바뀌는 경우(5개), 어미가 바뀌는 경우(2개), 어간과 어미가 모두 바뀌는 경우(1개)로 총 8개의 예문으로 제한한다. 예문을 komoran과 okt에 각각 입력하여 결과를 비교하고, 각 분석기의 메모리 사용의 효율성을 비교한다. ‘지었다’, ‘걸었다’, ‘구워먹는다’, ‘흘러’, ‘퍼’, ‘연습하여라’, ‘이르러’, ‘파래’의 komoran과 okt의 형태소 분석 결과를 중점적으로 비교한다.

어간이 바뀌는 경우	‘ㅅ’ 불규칙	‘ㅅ’이 모음 어미 앞에서 탈락하는 현상	철수가 웃긴 표정을 지었다.
	‘ㄷ’ 불규칙	‘ㄷ’이 모음 어미 앞에서 ‘ㄹ’로 바뀌는 현상	철수가 숲길을 걸었다.
	‘ㅂ’ 불규칙	‘ㅂ’이 모음 어미 앞에서 ‘오/우’로 바뀌는 현상	철수가 고기를 구워 먹는다.
	‘ㄹ’ 불규칙	‘ㄹ’가 모음 어미 앞에서 ‘ㄹㄹ’로 바뀌는 현상	강물이 조용히 흘러 내려간다.
	‘우’ 불규칙	‘우’가 모음 어미 앞에서 탈락하는 현상	아이들은 자기 그릇에 국을 퍼 담기 시작했다.
어미가 바뀌는 경우	‘여’ 불규칙	어간이 ‘하’로 끝나는 용언 뒤에서 모음 어미 ‘-아’가 ‘-여’로 바뀌는 현상	매일 꾸준히 연습하여라.
	‘러’ 불규칙	어간이 ‘르’로 끝나는 용언 뒤에서 모음 어미 ‘어’가 ‘러’로 바뀌는 현상	봄이 이르러 꽃들이 만개하기 시작했다.
어간과 어미가 모두 바뀌는 경우	‘ㅎ’ 불규칙	‘g’으로 끝나는 어간에 ‘-아/어’가 오면, 어간의 일부인 g이 탈락하고 어미도 변하는 현상	하늘이 파래 보인다.

- (1) Python Anaconda3(64-bit)과 java JDK 1.8을 설치한다.
- (2) Python에서 새로운 가상환경을 생성하고 활성화한다.
- (3) Java JDK 1.8의 환경 변수를 올바르게 설정한다.
- (4) Python의 프롬프트에 명령어 **pip install konlpy**와 **pip install JPype1**을 순차적으로 입력한다. 이후 python 인터프리터를 실행한다.
pip install konlpy: python의 pip를 통해 konlpy를 설치한다.
pip install JPype1: python과 java간의 인터페이스를 제공하는 JPype1을 설치한다.¹⁷⁾

 17) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

```
(base) PS C:\Users\정지윤> pip install konlpy
Requirement already satisfied: konlpy in c:\konlpy\lib\site-packages (0.6.0)
Requirement already satisfied: JPype1>=0.7.0 in c:\konlpy\lib\site-packages (from konlpy) (1.5.0)
Requirement already satisfied: lxml>=4.1.0 in c:\konlpy\lib\site-packages (from konlpy) (4.9.3)
Requirement already satisfied: numpy>=1.6 in c:\konlpy\lib\site-packages (from konlpy) (1.26.4)
Requirement already satisfied: packaging in c:\konlpy\lib\site-packages (from JPype1>=0.7.0->konlpy) (23.1)
(base) PS C:\Users\정지윤> pip install JPype1
Requirement already satisfied: JPype1 in c:\konlpy\lib\site-packages (1.5.0)
Requirement already satisfied: packaging in c:\konlpy\lib\site-packages (from JPype1) (23.1)
(base) PS C:\Users\정지윤> python
Python 3.11.7 | packaged by Anaconda, Inc. | (main, Dec 15 2023, 18:05:47) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

(5) pos() 메소드를 사용하여 예문의 형태소 분석을 각각 수행한다.

```
(base) PS C:\Users\정지윤> python
Python 3.11.7 | packaged by Anaconda, Inc. | (main, Dec 15 2023, 18:05:47) [MSC v.1916
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
>>> from konlpy.tag import Komoran, Okt
>>>
>>> # Komoran 클래스 인스턴스 생성
>>> komoran = Komoran()
>>> # 문장분석
>>> komoran_result = komoran.pos("철수가 웃긴 표정을 지었다.")
>>> print(komoran_result)
[('철수', 'NNP'), ('가', 'JKS'), ('웃기', 'VV'), ('ㄴ', 'ETM'), ('표정', 'NNG'), ('을',
'JKO'), ('지', 'VX'), ('었', 'EP'), ('다', 'EF'), ('.', 'SF')]
>>>
>>> # Okt 클래스 인스턴스 생성
>>> okt = Okt()
>>> okt_result = okt.pos("철수가 웃긴 표정을 지었다.")
>>> print(okt_result)
[('철수', 'Noun'), ('가', 'Josa'), ('웃긴', 'Verb'), ('표정', 'Noun'), ('을', 'Josa'),
('지었다', 'Verb'), ('.', 'Punctuation')]
```

<스 불규칙의 예문 '철수가 웃긴 표정을 지었다'의 형태소 분석>

4.2 분석 결과 정리

	komoran 클래스
‘스’ 불규칙	[('철수', 'NNP'), ('가', 'JKS'), ('웃기', 'VV'), ('ㄴ', 'ETM'), ('표정', 'NNG'), ('을', 'JKO'), ('지', 'VX'), ('엿', 'EP'), ('다', 'EF'), ('.', 'SF')]
‘ㄷ’ 불규칙	[('철수', 'NNP'), ('가', 'JKS'), ('숨길', 'NNP'), ('을', 'JKO'), ('걸', 'VV'), ('엿', 'EP'), ('다', 'EF'), ('.', 'SF')]
‘ㅂ’ 불규칙	[('철수', 'NNP'), ('가', 'JKS'), ('고기', 'NNG'), ('를', 'JKO'), ('굽', 'VV'), ('어', 'EC'), ('먹', 'VX'), ('는다', 'EF'), ('.', 'SF')]
‘르’ 불규칙	[('강물', 'NNG'), ('이', 'JKS'), ('조용히', 'MAG'), ('흐르', 'VV'), ('어', 'EC'), ('내려가', 'VV'), ('ㄴ다', 'EF'), ('.', 'SF')]
‘우’ 불규칙	[('아이들', 'NNP'), ('은', 'JX'), ('자기', 'NP'), ('그릇', 'NNG'), ('에', 'JKB'), ('국', 'NNG'), ('을', 'JKO'), ('꾸', 'VV'), ('어', 'EC'), ('담', 'VV'), ('기', 'ETN'), ('시작', 'NNG'), ('하', 'XSV'), ('았', 'EP'), ('다', 'EF'), ('.', 'SF')]
‘여’ 불규칙	[('매일', 'MAG'), ('꾸준히', 'MAG'), ('연습', 'NNG'), ('하', 'XSV'), ('아라', 'EF'), ('.', 'SF')]
‘러’ 불규칙	[('봄이', 'NNP'), ('이르', 'VV'), ('어', 'EC'), ('꽃', 'NNG'), ('들', 'XSN'), ('이', 'JKS'), ('만개', 'NNG'), ('하', 'XSV'), ('기', 'ETN'), ('시작', 'NNG'), ('하', 'XSV'), ('았', 'EP'), ('다', 'EF'), ('.', 'SF')]
‘ㅎ’ 불규칙	[('하늘', 'NNG'), ('이', 'JKS'), ('파래', 'NNG'), ('보이', 'VV'), ('ㄴ다', 'EF'), ('.', 'SF')]

	okt 클래스
‘ㅅ’ 불규칙	[(‘철수’, ‘Noun’), (‘가’, ‘Josa’), (‘웃긴’, ‘Verb’), (‘표정’, ‘Noun’), (‘을’, ‘Josa’), (‘지었다’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘ㄷ’ 불규칙	[(‘철수’, ‘Noun’), (‘가’, ‘Josa’), (‘숲길’, ‘Noun’), (‘을’, ‘Josa’), (‘걸었다’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘ㅂ’ 불규칙	[(‘철수’, ‘Noun’), (‘가’, ‘Josa’), (‘고기’, ‘Noun’), (‘를’, ‘Josa’), (‘구워’, ‘Verb’), (‘먹는다’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘르’ 불규칙	[(‘강물’, ‘Noun’), (‘이’, ‘Josa’), (‘조용히’, ‘Adjective’), (‘흘러’, ‘Verb’), (‘내려간다’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘우’ 불규칙	[(‘아이’, ‘Noun’), (‘들’, ‘Suffix’), (‘은’, ‘Josa’), (‘자기’, ‘Noun’), (‘그릇’, ‘Noun’), (‘에’, ‘Josa’), (‘국’, ‘Noun’), (‘을’, ‘Josa’), (‘퍼’, ‘PreEomi’), (‘담기’, ‘Verb’), (‘시작’, ‘Noun’), (‘했다’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘여’ 불규칙	[(‘매일’, ‘Noun’), (‘꾸준히’, ‘Adjective’), (‘연습’, ‘Noun’), (‘하여라’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘러’ 불규칙	[(‘봄’, ‘Noun’), (‘이’, ‘Josa’), (‘이르러’, ‘Verb’), (‘꽃들이’, ‘Noun’), (‘만’, ‘Modifier’), (‘개’, ‘Noun’), (‘하기’, ‘Verb’), (‘시작’, ‘Noun’), (‘했다’, ‘Verb’), (‘.’, ‘Punctuation’)]
‘ㅎ’ 불규칙	[(‘하늘’, ‘Noun’), (‘이’, ‘Josa’), (‘파래’, ‘Noun’), (‘보인다’, ‘Verb’), (‘.’, ‘Punctuation’)]

5. 분석 결과

5.1 komoran 클래스 분석 결과

올바르게 분석된 불규칙 활용	‘ㄷ’ 불규칙, ‘ㅂ’ 불규칙, ‘르’ 불규칙, ‘우’ 불규칙, ‘여’ 불규칙, ‘러’ 불규칙
부정확하게 분석된 불규칙 활용	‘ㅅ’ 불규칙, ‘ㅎ’ 불규칙

komoran은 용언 ‘지었다’를 [‘지’, ‘었’, ‘다’, ‘.’]로 형태소 분석하였다. ‘ㅅ’ 불규칙을 부정확하게 분석하였고, 기본형 ‘짓다’의 어간 ‘짓-’을 분리하지 못하였다.

‘걸었다’는 [‘걸’, ‘었’, ‘다’, ‘.’]로 형태소 분석되었다. ‘ㄷ’ 불규칙이 올바르게 분석되었고, 기본형 ‘걷다’의 어간 ‘걸-’을 올바르게 분리하였다.

‘구워먹는다’는 [‘굽’, ‘어’, ‘먹’, ‘는다’, ‘.’]로 형태소 분석되어, ‘ㅂ’ 불규칙이 올바르게 분석되었다. 기본형 ‘구워먹다’의 어간 ‘구워먹-’을 [‘굽’, ‘어’, ‘먹’]으로 올바르게 분석하였다. 그러나 ‘는다’를 하나의 형태소로 분석해 시제 선어말 어미 ‘느’를 분리하지 못하였다.

‘흘러’는 [‘흐르’, ‘어’, ‘.’]로 형태소 분석되었다. ‘르’ 불규칙이 올바르게 분석되었고, 기본형 ‘흐르다’의 어간 ‘흐르-’를 올바르게 분리하였다.

‘퍼’는 [‘푸’, ‘어’, ‘.’]로 형태소 분석되었다. ‘우’ 불규칙이 올바르게 분석되었고, 기본형 ‘푸다’의 어간 ‘푸-’를 올바르게 분리하였다.

‘연습하여라’는 [‘연습’, ‘하’, ‘아라’, ‘.’]로 형태소 분석되었다. ‘여’ 불규칙이 올바르게 분석되었고, 기본형 ‘연습하다’의 어간 ‘연습하-’를 [‘연습’, ‘하’]으로 올바르게 분석하였다.

‘이르러’는 [‘이르’, ‘어’, ‘.’]로 형태소 분석되었다. ‘러’ 불규칙이 올바르게 분석되었고, 기본형 ‘이르다’의 어간 ‘이르-’를 올바르게 분리하였다.

‘파래’ [‘파래’, ‘.’]로 형태소 분석되었다. ‘ㅎ’ 불규칙을 부정확하게 분석하였고, 기본형 ‘파랳다’의 어간 ‘파랳-’을 분리하지 못하였다.

5.2 okt 클래스 분석 결과

올바르게 분석된 불규칙 활용	없음
부정확하게 분석된 불규칙 활용	‘ㅅ’ 불규칙, ‘ㄷ’ 불규칙, ‘ㅂ’ 불규칙, ‘르’ 불규칙, ‘우’ 불규칙, ‘여’ 불규칙, ‘러’ 불규칙, ‘ㅎ’ 불규칙

Okt는 용언 ‘지었다’를 [‘지었다’, ‘.’]로 형태소 분석하였다. ‘ㅅ’ 불규칙을 부정확하게 분석하였고, 기본형 ‘짓다’의 어간 ‘짓-’을 분리하지 못하였다.

‘걸었다’는 [‘걸었다’, ‘.’]로 형태소 분석되었다. ‘ㄷ’ 불규칙이 부정확하게 분석되었고, 기본형 ‘걷다’의 어간 ‘걸-’을 분리하지 못하였다.

‘구워먹는다’는 [‘구워’, ‘먹는다’, ‘.’]로 형태소 분석되어, ‘ㅂ’ 불규칙이 부정확하게 분석되었다. 기본형 ‘구워먹다’의 어간 ‘구워먹-’을 분리하지 못하였다.

‘흘러’는 [‘흘러’, ‘.’]로 형태소 분석되어, ‘르’ 불규칙이 부정확하게 분석되었고, 기본형 ‘흐르다’의 어간 ‘흐르-’를 분리하지 못하였다.

‘퍼’는 [‘퍼’, ‘.’]로 형태소 분석되어, ‘ㅂ’ 불규칙이 부정확하게 분석되었고, 기본형 ‘푸다’의 어간 ‘푸-’를 분리하지 못하였다.

‘연습하여라’는 [‘연습’, ‘하여라’, ‘.’]로 형태소 분석되어, ‘여’ 불규칙이 부정확하게 분석되었고, 기본형 ‘연습하다’의 어간 ‘연습하-’를 분리하지 못했다.

‘이르러’는 [‘이르러’, ‘.’]로 형태소 분석되어, ‘러’ 불규칙이 부정확하게 분석되었고, 기본형 ‘이르다’의 어간 ‘이르-’를 분리하지 못했다.

‘파래는’ [‘파래’, ‘.’]로 형태소 분석되어, ‘ㅎ’ 불규칙이 부정확하게 분석되었고, 기본형 ‘파랳다’의 어간 ‘파랳-’을 분리하지 못했다.

5.3 komoran과 okt의 분석결과 비교

예문의 형태소 분석을 수행했을 때, komoran은 ‘ㄷ’ 불규칙, ‘ㅂ’ 불규칙, ‘르’ 불규칙, ‘우’ 불규칙, ‘여’ 불규칙, ‘러’ 불규칙을 올바르게 분석하였고, okt는 용언의 불규칙 활용을 전부 부정확하게 분석하였다. 따라서 본 연구를 통해 komoran이 okt보다 불규칙 용언의 형태소 분석 성능이 우수함이 입증되었다.

Okt는 다수의 용언의 어간과 어미를 올바르게 분리하지 못하였으며, 용언의 활용형 분석 정확도가 비교적 낮다. Okt는 후속단계에서 이를 수정하기 위한 추가적인 연산들이 누적되면서 메모리를 비효율적으로 사용한다는 단점이 있다. 예를 들어, 텍스트 전처리 과정에서 텍스트 단위당 메모리의 소모량이 증가한다.¹⁸⁾ 형태소 분석 이후 후속단계인 정제 및 추출, 인코딩을 거쳐 용언의 활용형이 한 단어로 저장되고 처리되는 것이 통계적 분석, 정보 검색 등의 자연어 처리 기술에서 메모리 사용의 효율성을 높이기 위해 중요하다.¹⁹⁾ 그러나 부정확한 형태소 분석으로 인해 용언의 활용형이 각기 다른 단어로 인식되어 여러 번 저장되고 처리되기도 한다.²⁰⁾ 이는 메모리 내에서 불필요한 공간을 차지하며, 자연어 처리 기술의 전반적인 성능 저하를 유발한다.

6. 결론

첫째, 본 연구는 실험을 통해 komoran이 okt에 비해 불규칙 용언의 처리 성능이 우수하고, 메모리를 효율적으로 사용한다는 점을 입증하였다.

둘째, komoran은 용언 처리의 정확도와 메모리의 효율성 측면에서 대용량의 데이터를 처리하는데 유용하다. komoran은 불규칙한 언어 패턴을 비교적 정확하게 분석하고, 메모리 리소스를 효율적으로 사용한다. 따라서 komoran은 대용량의 데이터를 처리하는데 유용하며, 통계적 분석, 정보 검색과 같은 자연어 처리 기술에 효율적으로 사용될 수 있다.

셋째, okt는 대용량의 데이터 처리에 부적합하고, 개선이 필요하다. Okt는 경량 분석기로써, 소셜 미디어 텍스트나 간단한 텍스트 처리에 강점이 있지만, 대용량의 데이터 처리에는 부적합하다. 이는 시간이 필요하지만, okt의 학습 데이터의 양과 종류를 늘리는 방안을 통해 개선 가능하다.

18) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

19) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

20) OpenAI(2022). ChatGPT(November 30 version) [Large language model],
<https://openai.com/index/chatgpt>

* 첨부

	ChatGPT 프롬프트 및 수정 사항
서론	없음
본론	외각주 (9) 발표 후에 한 청중으로부터 ‘텍스트 전처리에서 왜 형태소 분석을 수행하는가?’에 대한 질문을 받았다. 필자는 형태소 분석이 텍스트 전처리에서 가장 기초적인 단계라고 생각해왔기에 그 이유를 깊이 고민해보지 않았다. 그러나 청중의 질문 이후, 그 이유가 궁금해져 chatGPT를 통해 알아보았고, 관련 내용을 ‘2.2 텍스트 전처리의 중요성’ 부분에 추가하였다. 명령 프롬프트: ‘한국어 텍스트 전처리에서 토큰화를 형태소 단위로 수행하는 이유가 무엇이야?’ (14,15,16) 논문을 여러개 찾아보았으나 komoran과 okt에 대한 신뢰성 있는 자료를 찾기가 쉽지 않아 chatGPT를 통해 komoran과 okt에 대해 조사한 자료를 참고하였다. 명령 프롬프트: ‘komoran과 okt의 차이에 대해 설명해’/ ‘komoran에서 사용 가능한 메소드들과 그 특징에 대해 설명해’ · ‘러’ 불규칙 예문이 떠오르지 않았다. 명령 프롬프트: “‘이르러’라는 단어를 포함한 예문을 생성해” (17) 명령 프롬프트: ‘pip install konlpy/ pip install Jpype1 명령어의 역할을 설명해’ (18) 수정 전: 예를 들어, 한 텍스트 단위당 메모리의 소모량이 많다. 수정 후: 예를 들어, 텍스트 전처리 과정에서 텍스트 단위당 메모리의 소모량이 증가한다. (19) 수정 전: 형태소 분석 이후 후속단계인 정제 및 추출, 인코딩을 거쳐 용언의 활용형이 한 단어로 저장되고 처리되는 것이 통계적 분석, 정보 검색 등의 자연어 처리 기술에서 중요하다. 수정 후: 형태소 분석 이후 후속 단계인 정제, 추출, 인코딩을 통해 용언의 활용형이 한 단어로 저장되고 처리되는 것이 통계적 분석, 정보 검색 등의 자연

	어 처리 기술에서 메모리 사용 효율성을 높이는 데 중요하다.
결론	· chatGPT를 참고하여 수정 전 ‘언어 패턴, 메모리’에서 각각 ‘불규칙한 언어 패턴, 메모리 리소스’로 수정하였다. · 명령 프롬프트: ‘okt의 성능을 향상시킬 수 있는 방법은?’
참고 문헌	자연어 처리 (naver.com) 챗봇-Google검색 김철희 외, 「한국어 교육을 위한 형태소 분석기의 조건과 향후 과제」, 한국국어교육학회, 2021 최아리, 「자연어처리 기술을 활용한 국내 형사 판결문 검색 모델 비교 연구」, 한림대학교 대학원, 2022 한국정보통신기술협회, 「지식백과 자연어 처리」, 2024년 6월 8일 검색 https://terms.naver.com/entry.naver?docId=5917571&cid=66682&categoryId=66682 황현선 「한국어 자연어처리를 위한 딥러닝 모델의 속도 향상」, 강원대학교 대학원, 2022 구혜승, 「한국어 에세이 자연어 처리를 위한 한국어 형태소 분석기 개선 방안」, 한국교원대학교 대학원, 2024 서울대학교 AI연구원, 「지식백과 형태소 분석기」, 2024년 6월 10일 검색, https://terms.naver.com/entry.naver?docId=6653681&cid=69974&categoryId=69974 유원준, 「딥 러닝을 이용한 자연어 처리 입문」 e-book, 2024

